MagicCode Super-Resolution Software Release Notes

Document Version: v3.0.0. Release Date: October 24, 2026.

1. Summary

MagicCode Super-Resolution Software v3.0.0 (since v2.2.1) publishes renamed SR alg_mode_e values, a two-frame mc_nscaler_enable ABI, and Vulkan frame generation (GEN). Image and graphic SR remain on the nscaler API. FG is Vulkan-only in builds with MAGIC_SR_HAS_FG (Windows and Linux product libraries). Android Vulkan builds do not define MAGIC_SR_HAS_FG. D3D11 / Metal / GLES / OpenGL / CPU return -100050 / -102013 for GEN.

2. Key Changes

v3.0.0 (October 24, 2026; includes changes since v2.2.1, including the intermediate v2.2.2 Windows archive rebuild):

- Public version string is v3.0.0 (mc_nscaler_version).
- Breaking: mc_nscaler_enable takes two input-frame pointers: int mc_nscaler_enable(void **handle, mc_nscaler_input_frame_t *in_frame[2], mc_nscaler_output_frame_t *out_frame). Non-GEN callers pass mc_nscaler_input_frame_t *in2[2] = { &in, NULL }. in_frame[1] is read only when a GEN_* mode is selected; other modes must not read it.
- Breaking: alg_mode_e renamed. SPATIAL_SPEED_MODE → SR_SPEED_MODE_FOR_IMAGE (0); SPATIAL_BALANCED_MODE → SR_BALANCED_MODE_FOR_IMAGE (1); TEMPORAL_SPEED_MODE → SR_SPEED_MODE_FOR_GRAPHIC (2); TEMPORAL_BALANCED_MODE → SR_BALANCED_MODE_FOR_GRAPHIC (3). Numeric 0..3 keep the same SR meaning. MAX_ALG_MODE is 16.
- New GEN bits: GEN_SPEED_MODE_FOR_GRAPHIC = 1<<2 (4, FG-Fast, analytic, no weights) and GEN_BALANCED_MODE_FOR_GRAPHIC = 1<<3 (8, FG-CoreV3). SR and GEN may be OR-ed (example: 2|4 = 6). Both GEN bits together are invalid (MAX_ALG_MODE=16; do not set both bits of MC_ALG_GEN_MASK).
- Frame generation: a GEN handle with in_frame[1]==NULL uses the existing SR path; with in_frame[1]!=NULL it interpolates t=0.5 between in_frame[0] and in_frame[1] into out_frame. Color is output resolution; depth and motion stay at render resolution. Commands record into in_frame[0].command_buffer (Vulkan only; no submit/wait). After return the output image is VK_IMAGE_LAYOUT_GENERAL. in_frame[0].width/height must equal the handle's render input size or the library recreates the handle.
- FG weights (GEN_BALANCED): ctrl_param_t.model_path is the FG weight file path. Empty path uses MAGIC_FG_WEIGHT_PATH, then magic_fg_core.bin in the working directory, then model/magic_fg_core.bin. Packaged file: model/magic_fg_core.bin (331,136 bytes). model_path

is immutable; changing weights requires recreating the handle. GEN_SPEED needs no weights. Errors: -100051 / -102014 (weights missing or invalid), -100052 / -102015 (FG create failed), -100050 / -102013 (GEN not available). Public MC_ERROR_* codes are in mc_error.h (included by mc_interface.h).

- mc_nscaler_version() and mc_build_id() return pointers into the loaded module; copy the bytes before FreeLibrary/dlclose. The Windows product DLL exports six C symbols: mc_nscaler_control, mc_nscaler_disable, mc_nscaler_enable, mc_nscaler_version, mc_build_id, mc_loaded_module_path (build/windows/magic_sr.def).
- Windows product libraries: lib/windows/libmagic_sr.dll, libmagic_sr.lib, libmagic_sr.dll.lib. Linux: lib/linux/libmagic_sr.a and libmagic_sr.so (CPU image + Vulkan Graphic (Vulkan Temporal)/TAAU + FG).
- Vulkan Speed (Lite-TAAU) now lets an explicit strong reactive mask suppress the still-camera accumulation branch. Previously that branch ignored the caller reactive mask when the camera was still or per-pixel MV was below about 0.2 px, so pixels with ~0 MV but changing content (TV / video-wall / LED-screen textures) could keep ghosting for about 64–96 frames after a content cut. Strong reactive (≥ about 0.85, fully effective at ≥ 0.95) now weakens that accumulation. Omitting reactive or passing a weak value (< 0.85) is unchanged. Recommendation: mark screen-like objects whose MV is 0 but whose content changes (for example with CustomDepth stencil) as reactive at strength 1.0. Limit: Vulkan only in this release. The Metal / GLES / D3D11 Speed still-accumulation structure is the same idea but is not modified or verified here.

v2.2.0 (September 24, 2026; documentation refresh September 30, 2026):

- Public version string is v2.2.0 (mc_nscaler_version).
- Public API is mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable / mc_nscaler_version. It replaces the v2.0.0 MC_Init / MC_Process / MC_Uninit / MC_Control entry and the v2.1.0 MC_Enable / MC_Disable helper; mc_enable.h is removed.
- ctrl_param_t replaces input_param_t (no struct_size/width/height; the image size moves to mc_nscaler_input_frame_t / mc_nscaler_output_frame_t, which replace magic_frame_t).
- Fixed a Vulkan Balanced 1.5x SIGSEGV (FSR1 timestamp query pool creation).
- Linux x86_64 platform library: lib/linux/libmagic_sr.a (CPU image, AVX2/FMA; rebuild via tools/build_linux_lib.sh). The archive carries no GPU backend objects; Linux Vulkan Graphic (Vulkan Temporal) builds from source and is validated headless (lavapipe) with output matching the Windows NVIDIA reference.
- Image (1,2) open band: BALANCED routes 2dir LUT then lanczos2HV; SPEED routes 2dir LUT then bilinear. The lut_boost last convolution treats out-of-image taps as zero.

v2.0.0 (September 2, 2026) key changes:

- Public version string is v2.0.0 (MC_GetVersion).
- alg_mode_e: SPATIAL_SPEED_MODE=0, SPATIAL_BALANCED_MODE=1, TEMPORAL_SPEED_MODE=2, TEMPORAL_BALANCED_MODE=3.
- Graphic scaler range is (1, 8]. Image remains [1.0, 8.0] subject to backend and model.

- Formal graphic entry is MC_Init / MC_Process / MC_Uninit with magic_frame_t + temporal_frame_t (color/output, depth, motion, jitter, frame_index/reset_history, camera, gpu_context, command-buffer).
- MC_Enable still processes one color texture and is not a complete graphic entry.
- Windows product archives: lib/windows/libmagic_sr.lib and libmagic_enable_sr.lib. Enable does not accept MAGIC_BACKEND_D3D11.
- Both graphic modes default to deriving a missing reactive mask (and approximate transparency on the Metal/Vulkan/GLES Speed path) and to depth+MV history rejection. Explicit masks win per channel.

3. Compatibility

Platform	GPU backends	Graphic
Android	Vulkan, OpenGLES	Graphic GPU yes; CPU graphic no; FG not in this Android Vulkan build
iOS / iPadOS / macOS	Metal	Graphic GPU yes; CPU graphic no; FG not available (Metal)
Windows	Vulkan, Direct3D 11, OpenGL 4.3	Graphic GPU yes; CPU graphic no; FG on Vulkan (MAGIC_SR_HAS_FG)
Linux x86_64	Vulkan	CPU image + Vulkan Graphic (Vulkan Temporal)/TAAU + FG in libmagic_sr.a/.so; CPU graphic no

- Operating systems: Windows 10 64-bit and above, Android 8.0 and above, iOS 13.0 and above, iPadOS 13.0 and above, macOS 12.0 and above, and Linux x86_64 (glibc). FG is Vulkan-only and requires MAGIC_SR_HAS_FG; Android Vulkan builds currently omit FG.
- Input resolution: 64×64 to 4032×4032.
- CPU SIMD (image CPU paths): AVX2 on x86_64, NEON on ARM64. CPU graphic is unavailable.

Graphic Super-Resolution

Graphic super-resolution runs through the nscaler session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable. v2.2.0 replaced the v2.0.0 MC_Init / MC_Process / MC_Uninit entry and removed the v2.1.0 MC_Enable helper (mc_enable.h is deleted). A NULL *handle on mc_nscaler_enable creates a session with defaults (2x SR_SPEED_MODE_FOR_IMAGE) and processes the first frame. Pass mc_nscaler_input_frame_t *in2[2] = { &in, NULL } (set in2[1] only for GEN interpolate).

Modes

Value	Enumerator	Role
0	SR_SPEED_MODE_FOR_IMAGE	Image, throughput first
1	SR_BALANCED_MODE_FOR_IMAGE	Image, quality/cost trade-off
2	SR_SPEED_MODE_FOR_GRAPHIC	Graphic, throughput first

3	SR_BALANCED_MODE_FOR_GRAPHIC	Graphic, canonical FSR-family path
4	GEN_SPEED_MODE_FOR_GRAPHIC	FG-Fast (analytic, no weights). Vulkan + MAGIC_SR_HAS_FG. Combinable with SR bits; bare 4 equals SR_SPEED_MODE_FOR_IMAGE|GEN_SPEED.
8	GEN_BALANCED_MODE_FOR_GRAPHIC	FG-CoreV3. Same availability as Fast. Needs magic_fg_core.bin. Do not set both GEN bits.

- Graphic scaler_factor is (1, 8]; 1.0 itself is rejected. Image remains [1.0, 8.0].
- SR_SPEED_MODE_FOR_GRAPHIC on Metal, Vulkan, and OpenGLES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family graphic path (same family as Balanced, not a separate customer API).
- SR_BALANCED_MODE_FOR_GRAPHIC uses the canonical FSR-family path on every GPU graphic backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLES do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run graphic modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- mc_nscaler_input_frame_t / mc_nscaler_output_frame_t: caller-owned color and output GPU resources.
- mc_nscaler_input_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at create (mc_nscaler_control SET_PARAM, or the first mc_nscaler_enable): Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan graphic requires mc_nscaler_input_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on ctrl_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.

- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both graphic modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

4. Migration Notes

From v2.x to v3.0.0:

- Change every mc_nscaler_enable call to pass a two-pointer array. For SR-only: in2[0]=∈ in2[1]=NULL.
- Rename alg_mode enumerators as in the mapping above. Values 0..3 are unchanged; add GEN bits only when integrating FG.
- If using GEN_BALANCED, set ctrl_param_t.model_path to the FG weight file, or leave it empty and ship model/magic_fg_core.bin (or set MAGIC_FG_WEIGHT_PATH). Recreate the handle to change weights.
- Copy mc_nscaler_version() / mc_build_id() before unloading the DLL.
- See the Migration Guide for before/after samples and common errors.

From v2.0.0 / v2.1.0 to v2.2.0:

- Rename MC_Init to mc_nscaler_control(MC_NSCALER_CMD_SET_PARAM) create (or NULL-handle mc_nscaler_enable), MC_Process to mc_nscaler_enable, MC_Uninit to mc_nscaler_disable, MC_GetVersion to mc_nscaler_version.
- Replace input_param_t with ctrl_param_t (no struct_size/width/height) and magic_frame_t with mc_nscaler_input_frame_t / mc_nscaler_output_frame_t.
- MC_Enable and mc_enable.h are removed; default-create covers the one-texture image case.

v2.0.0 migration notes (historical):

- From v1.2.x: keep numeric 0/1 for image modes (now named SPATIAL_*). Add 2/3 only when integrating graphic.
- Replace any MC_Process(handle, image_in, image_out) usage with MC_Process(handle, magic_frame_t*).
- Set input_param_t.struct_size and temporal_frame_t.struct_size.
- Do not treat Enable graphic enums as a full graphic integration.
- Review Privacy Policy and Data Reporting Notice before distributing applications that include the SDK or plugins. Those legal documents are unchanged by this technical release.

5. Known Limitations

- FG is Vulkan only. Android Vulkan builds do not define MAGIC_SR_HAS_FG; GEN create returns -100050 / -102013. D3D11 / Metal / GLES / OpenGL / CPU likewise.
- GEN_BALANCED requires a readable FG weight file via model_path (or the default search). GEN_SPEED does not.
- GEN in_frame[0].width/height must match the handle render input size; a mismatch recreates the handle (mc_nscaler_enable).
- Vulkan Speed (Lite-TAAU) now lets an explicit strong reactive mask suppress the still-camera accumulation branch. Previously that branch ignored the caller reactive mask when the camera was still or per-pixel MV was below about 0.2 px, so pixels with ~0 MV but changing content (TV / video-wall / LED-screen textures) could keep ghosting for about 64–96 frames after a content cut. Strong reactive (≥ about 0.85, fully effective at ≥ 0.95) now weakens that accumulation. Omitting reactive or passing a weak value (< 0.85) is unchanged. Recommendation: mark screen-like objects whose MV is 0 but whose content changes (for example with CustomDepth stencil) as reactive at strength 1.0. Limit: Vulkan only in this release. The Metal / GLES / D3D11 Speed still-accumulation structure is the same idea but is not modified or verified here.
- Not every backend, scale, or device class is device-run. Distinguish API support from verified configurations.
- Derived masks are approximations. Omitting transparency is not equivalent to a semantic transparency mask.
- Vulkan graphic requires a recording command buffer and explicit layouts; the library does not begin, end, or submit it.
- Linux product libraries include CPU image plus Vulkan Graphic (Vulkan Temporal)/TAAU and FG.
- Performance depends on device, driver, backend, resolution, scale, and motion quality.

6. Validation Snapshot

API contracts are defined by interface/mc_interface.h. Public MC_ERROR_* codes are in interface/mc_error.h (included by mc_interface.h). Rate coverage in tests is not a device-run of every rate. Do not treat historical v2.2.0 pass counts as v3.0.0 results.